ROADMAP AND HOW-TO GUIDE

KIIT Transit: what comes next

The remaining work to take the app from a tested prototype to a release on students' phones, with instructions for each task.

AUDIENCE
Developers and project team

REPOSITORY
github.com/LostRunes/kiit-transit

UPDATED
23 October 2026

01 The work at a glance

The app runs end to end in demo mode. What remains is connecting it to a real backend, putting it on real phones, correcting map data and closing a few gaps before release.

Tasks are listed in the order they should be done. Items 2 to 5 are needed before any student uses the app; the rest can follow.

#	Task	Priority	Effort
2	Set up the project on a new machine	First	Under 1 hour
3	Build and install the Android app	Before release	1 to 2 hours
4	Connect the real Supabase backend	Before release	Half a day
5	Field test with drivers and students	Before release	1 to 2 days
6	Correct campus and stop positions	High	1 to 2 days
7	Protect the map key	High	30 minutes
8	Prepare the release build	Before release	Half a day
9	Add timetables when available	Medium	Depends on data
10	Remaining design screens	Medium	2 to 4 days
11	Phase 2: dashboard, notifications, Google sign-in	Later	1 to 2 weeks

02 Set up the project on a new machine

You need Flutter 3.38 or newer, Git, Android Studio (for the Android SDK and an emulator) and Python 3.10 or newer for the map-data tool. Run `flutter doctor` and fix anything it reports before continuing.

1. Clone the repository and fetch the Flutter packages.

```
git clone https://github.com/LostRunes/kiit-transit.git
cd kiit-transit
flutter pub get
copy .env.example .env      # Windows; use cp on macOS/Linux
```

Open `.env` and put the MapTiler key after `MAPTILER_KEY=`. Ask the project owner for it: keys are kept out of the repository and git ignores `.env`. Without a key the map still works with plain OpenStreetMap tiles.

2. Run the app in Chrome. While the Supabase values in `.env` are empty, it starts in demo mode with simulated vehicles.

```
flutter run -d chrome --dart-define-from-file=.env
```

Sign in with any @kiit.ac.in address and a password of six or more characters. `driver@kiit.ac.in` opens the driver app.

3. Check that the code is healthy before you change anything.

```
flutter analyze
flutter test
```

> **Done when**
> The login screen appears, demo vehicles move on the map, the analyser reports no issues and all 21 tests pass.

03 Build and install the Android app

Driver tracking only works properly on a real Android phone, because it relies on GPS and a background service. Start with a debug build.

1. On the phone, open Settings, About phone, and tap Build number seven times to enable Developer options. Then turn on USB debugging in Developer options.
2. Connect the phone by USB, accept the prompt on the phone, and confirm Flutter can see it with `flutter devices`.
3. Install and launch the app directly, or build an installer file to share.

```
flutter run --dart-define-from-file=.env            # installs and opens the app
flutter build apk --debug --dart-define-from-file=.env   # or build an installer
adb install build/app/outputs/flutter-apk/app-debug.apk
```

When the driver app first starts duty, Android asks for location access and notification permission. Choose "Allow all the time" for location, otherwise tracking stops when the screen is off.

> **Done when**
> The app opens on the phone, a demo driver can start duty, and the position keeps updating with the screen locked.

04 Connect the real Supabase backend

Until this step, all data is simulated and nothing is saved. Supabase provides the database, sign-in and live updates. The free tier is enough for testing.

1. Create an account at supabase.com and a new project. Choose the Mumbai (ap-south-1) region for the lowest delay from Bhubaneswar, and store the database password safely.

2. Open SQL Editor and run the whole of `supabase/schema.sql`. This creates the tables, security rules, live-update feeds and the function that merges ride requests.
3. Run `supabase/seed.sql` the same way. It adds the live campus routes, the vehicles and sample bus schedules.
4. Under Authentication, Providers, make sure Email is enabled. For a first test you can turn off "Confirm email" so new accounts work immediately; turn it back on before release.
5. Under Project Settings, API, copy the Project URL and the anon (publishable) key. Never use the service_role key in the app.
6. Put both values in `.env` and run the app. They are passed in at build time and never stored in the code.

```
SUPABASE_URL=https://<project-id>.supabase.co     # in .env
SUPABASE_ANON_KEY=<anon-key>

flutter run --dart-define-from-file=.env
```

7. Create accounts for the drivers through the app, then give them the driver role in SQL Editor. Repeat for each driver, with their phone number for the directory.

```
update profiles set role = 'driver', phone = '+91XXXXXXXXXX'
where email = 'driver.name@kiit.ac.in';
```

> **Keep the two route lists in step**
> The demo backend reads campus routes from `lib/data/datasources/seed_data.dart`, the real backend from `supabase/seed.sql`. If you change stops or IDs in one, change the other the same way.

> **Done when**
> A new student account can sign in, a promoted driver sees the driver app, and starting duty on one phone makes the vehicle appear on another phone within a few seconds.

05 Field test with drivers and students

Use at least two phones: one driver and one student. Test on the campus roads, not at a desk, and note anything that looks wrong.

Check	How	Expected result
Background tracking	Start duty, lock the driver phone, drive for 10 minutes.	The vehicle keeps moving on the student phone with no gaps over 15 seconds.

Check	How	Expected result
Battery saver	Repeat with battery saver on. If tracking stops, exclude the app from battery optimisation.	Tracking continues, or the setting to fix it is known.
Arrival estimates	Compare the shown arrival time with the real arrival at three stops.	Within a few minutes. Estimates use straight-line distance at 15 km/h.
Offline vehicles	End duty, or switch off mobile data for 3 minutes.	The vehicle disappears from the student map after 2 minutes.
Ride requests	Send a request from the student phone near a driver.	The driver sees it, with the waiting count, and can mark it served.
Trip planner	Plan five real journeys students make, for example hostel to Campus 25.	Sensible options; note any route that is missing or wrong.

> **Done when**
> All six checks pass, and a short list of problems found has been recorded.

06 Correct campus and stop positions

Many positions on the map are estimates. Only Campus 3, 6 and 15 are exact. Of the 383 Mo Bus stops, about 120 have a confirmed position; the others are spaced along the route between them and are marked "approximate" in the app. Stop order is correct; positions can be off by a few hundred metres.

Getting a correct coordinate

Open Google Maps, long-press the exact spot (the campus gate, or where the bus actually stops) and copy the two numbers shown, for example `20.35307, 85.82015`. The first is latitude, the second longitude.

Where each kind of position lives

What	File	What to edit
Campus landmarks on the live map	`lib/core/constants/campus_locations.dart`	Entries marked `// approx` (Campus 14, 17, 25, Food Court, Queens Hostels).
KIIT campuses in the route network	`tool/network_data.py`	The `CAMPUSES` table. Set the coordinates and change the last value from `True` to `False`.
Mo Bus stops	`tool/network_data.py`	Add the stop to `FIXED` with its exact coordinates, using the stop name exactly as written in the route lists.

Rebuilding the network after edits

1. Save your changes to `tool/network_data.py`.
2. Run the build script. It reuses saved lookups, so it finishes in a few minutes, and prints how many stops are still approximate.
3. Run the tests, then open the app, go to Routes and check the route you changed on the map.

```
python tool/build_network.py
flutter test
```

Work route by route. The busiest corridors are the best place to start: KIIT Square to Jaydev Vihar, Jaydev Vihar to the railway station, and Fire Station Square to Khandagiri.

> **Faster option: OpenStreetMap bus stops**
> OpenStreetMap has many named bus stops in Bhubaneswar. They can be downloaded with the Overpass API in small areas (large requests are refused). Match them to stop names and add the matches to `FIXED`. Keep requests small and spaced out, and check with the team before running a large download.

> **Done when**
> Every campus is marked exact, and the build script reports few or no approximate stops on the main corridors.

07 Protect the map key

The MapTiler key lives only in `.env`, which git ignores, and both the app and the map-data tool read it from there. The current key was briefly published in the public repository before that change, so treat it as exposed and replace it.

1. Sign in at cloud.maptiler.com, open Account, API keys, and create a new key.
2. Restrict it: add the app's Android package name and the web domain the app will be hosted on as allowed origins.
3. Put the new key in `.env` on every machine that builds the app, then revoke the old key.

```
MAPTILER_KEY=<new-key>          # in .env
flutter run --dart-define-from-file=.env
python tool/build_network.py    # reads the same .env
```

> **Done when**
> The old key is revoked, the new one is restricted, and the map still loads in the app.

08 Prepare the release build

A few settings still carry Flutter's defaults and must change before the app goes to the Play Store.

1. Change the application ID from `com.example.kiit_transit` to a permanent one, for example `in.ac.kiit.transit`, in `android/app/build.gradle.kts` (both `namespace` and `applicationId`). It cannot change after the first upload.
2. Create a signing key with `keytool`, store it outside the repository, and reference it from an `android/key.properties` file that is never committed.
3. Replace the debug signing in the `release` build type with the new key.
4. Set the app name and icon, then build the bundle for the Play Store.

```
flutter build appbundle --release --dart-define-from-file=.env
```

Google Play asks why the app uses location in the background. Explain that drivers share their vehicle's position during duty so students can see it, and include a short video of the driver duty screen.

> **Done when**
> A signed release bundle installs on a test phone and connects to the real backend.

09 Add timetables when they become available

Neither data source has departure times. KIIT vehicles leave when full, and the Mo Bus map lists stops only. The app says this openly instead of inventing times. Two ways to improve it:

- **Mo Bus:** ask CRUT (toll free 1800 345 0061) or check the Mo Bus app for first and last departures and the frequency of each route. Add these per route in `tool/network_data.py` as a new field, include it in the generated `network.json`, and show it on the route page and in the trip planner.
- **KIIT:** the backend currently keeps only each vehicle's latest position. Add a small log table that records when each duty starts and when a vehicle leaves its home campus. After a few weeks of use this gives real typical waits per route and time of day, which can replace the fixed "2 to 30 minutes" note.

> **Done when**
> At least the busiest Mo Bus routes show first bus, last bus and frequency.

10 Remaining design screens

The Google Stitch design (project 6481982968610420949) has four screens that were not built because they need features V1 lacks.

Screen	Needs	Where to add it
Onboarding	Nothing new; three intro slides shown once.	New page in `lib/views/common/`, shown before login on first launch.
Role selection	Decision on whether students pick a role or the admin assigns it.	Login flow in `lib/views/auth/`.
Notifications	Push notifications (task 11).	New student tab or a page opened from the Home bell.
Profile and stats	Trip history stored per student.	Extend `lib/views/common/profile_page.dart`.

Each new page gets a path in `lib/core/constants/route_paths.dart` and a route in `lib/core/routing/app_router.dart`. Use the shared building blocks in `lib/widgets/common.dart` (`Pod`, `SoftIcon`, `BrandBar`) so the screens match the rest of the app.

11 Phase 2

Admin dashboard

A web page for the transport office showing all vehicles, drivers on duty and where students are waiting, with history. The admin role already exists in the database. Build it as a separate Flutter web entry point that reuses the existing repositories.

Push notifications

Use Firebase Cloud Messaging. Store each phone's token in Supabase, and use a Supabase database trigger or Edge Function to notify nearby drivers when a ride request reaches a threshold, and students when a vehicle is close.

Google sign-in

Enable the Google provider in Supabase Authentication and restrict it to the kiit.ac.in domain, so students can sign in with their university Google account instead of a password.

12 Known limitations

Limitation	Why	Fix
Arrival times are rough	Straight-line distance at a fixed campus speed (V1 specification).	Use road distance along the route, as the trip planner already does.
Live campus loops draw straight lines	Only stops are stored for these routes.	Add intermediate points, or snap them to roads like the city network.
Trip planner times are estimates	Based on road distance and average speeds, not timetables.	Task 9.
Many map positions are approximate	Campuses and most Mo Bus stops are not in public map data.	Task 6.

13 Everyday commands

Command	What it does
`flutter run -d chrome --dart-define-from-file=.env`	Runs the app in a browser with the keys from `.env`.
`flutter analyze`	Checks the code for errors and style problems.
`flutter test`	Runs the 21 automated tests.
`dart format -l 130 lib test`	Formats the code to the project's line length.
`python tool/build_network.py`	Rebuilds the KIIT and Mo Bus network map data.
`flutter build apk --debug --dart-define-from-file=.env`	Builds an Android installer for testing.

The repository files `README.md` and `PROJECT_STATUS.md` hold the same information in shorter form and are kept up to date with each change.